mming Fundamentals

Week 7

Learning Outcomes:

- Students should be able to define repetitive statements
- Students should be able to repeat a Set of Instructions for a specific number of times using counter loop.
- Students should be able to repeat a Set of Instructions for an unknown number of times using conditional loop.
- Students should be able to solve larger problems by decomposing it into smaller sub-problems and combining their solution to achieve final results using the Counter Loop

Instructions

Use proper indentation to make your programs readable.

Use descriptive variables in your programs (Name of the variables should show their purpose)

Loop

Introduction

In real-world life, we are working with a repetitive process like bowler throwing six balls in an over, cash counting in banks, a wheel revolving its spindle. For this purpose, we need a structure that maps these methods into programming which is loops.

Loop:

A process which repeats itself again and again is called loop. In our daily life, we encounter many situations where we have to decide the repeated process.
For example

- Software of the ATM machine is in a loop to process transaction after transaction until you acknowledge that you have no more to do.
- Software program in a mobile device allows user to unlock the mobile with 5 password attempts.
- You put your favorite song on repeat mode.

Before implementing loops, let's recall the concepts that you were taught in the earlier class.

There are 2 types of Loops.

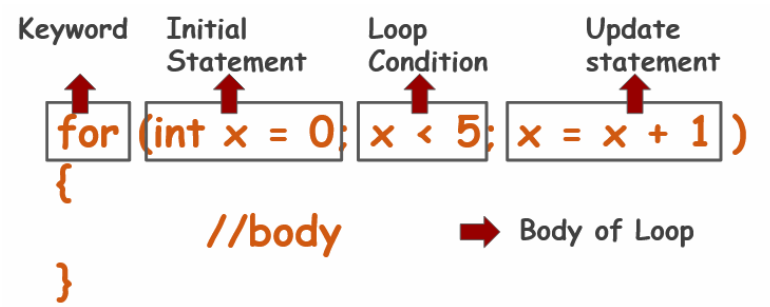
1. Counter Loops

2. Conditional Loops

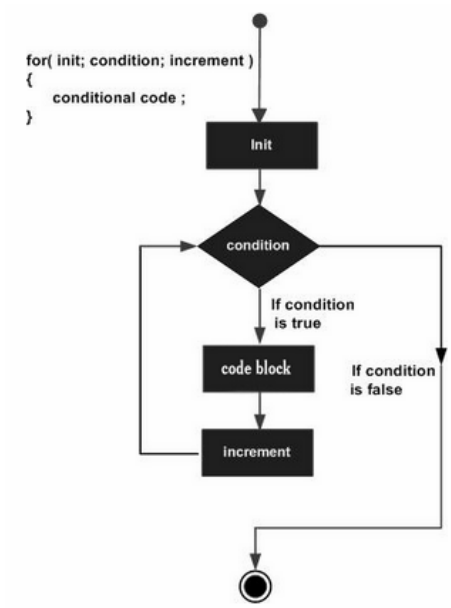
Counter Loop:

In Counter Loop, the program knows beforehand about how many times a specific instruction or set of instructions will be executed. In C++ programming language, **for loop** is used as a counter loop.

Syntax of For Loop:



Flow of For Loop:



Example #1:

Write a code showing that display counting from 1 to 10.

Solution

`#include <iostream>`
`using namespace std;`
`main()`
`{`
`  for(int i = 1; i<=10; i= i+1)`
`  {`
`  cout << i << endl;`
`  }`

```
}
```

The code produces the following output

```
1
2
3
4
5
6
7
8
9
10
```

Explanation:

Iteration	Variable	i <= 10	Action
1st	`i = 1`	`True`	`1` is printed. `i` is increased to `2`.
2nd	`i = 2`	`True`	`2` is printed. `i` is increased to `3`.
3rd	`i = 3`	`True`	`3` is printed. `i` is increased to `4`.
4th	`i = 4`	`True`	`4` is printed. `i` is increased to `5`.
5th	`i = 5`	`True`	`5` is printed. `i` is increased to `6`.
6th	`i = 6`	`True`	`6` is printed. `i` is increased to `7`.
7th	`i = 7`	`True`	`7` is printed. `i` is increased to `8`.
8th	`i = 8`	`True`	`8` is printed. `i` is increased to `9`.
9th	`i = 9`	`True`	`8` is printed. `i` is increased to `10`.
10th	`i = 10`	`True`	`10` is printed. `i` is increased to `11`.
11th	`i = 11`	`false`	The loop is terminated

Example 1 was an example of Independent Iterations which means the result of previous iteration was not required for next iterations. However, there are set of problems that require the result from previous iterations. Such iterations are called dependent iterations.
Let's see the Examples of Dependent Iterations.

Example #2:

Calculate the sum of first 5 natural numbers.

Remember: Before writing the program, first step is to identify the pattern within the sub problem such that this pattern can be written as a generic expression or set of generic expressions.

Solution

```
#include <iostream>
#include<iostream>
using namespace std;
main()
{
    int sum = 0;
    for(int i = 1; i<=5; i= i+1)
    {
        sum = sum + i;
    }
    cout << sum << endl;
}
```

The code produces the following output

```
15
```

Example # 3:

Write a code that prompts the user to input length of Fibonacci series and display the series.

Fibonacci Number Series

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, 4181, 6765, 10946, 17711, 28657, 46368, 75025, 121393, 196418, 317811...

Solution

```
#include <iostream>
using namespace std;
main()
{
    int n1 = 0, n2 = 1, next;
    int n;
    cout << "How many numbers of Fibonacci Series you want to print: ";
    cin >> n;

    cout << n1 << ", ";
    cout << n2;
    for(int x = 1; x < n - 1; x = x + 1)
    {
        next = n1 + n2;
```

```
        cout << ", " << next;
        n1 = n2;
        n2 = next;
    }
}
```

The code produces the following output

```
How many numbers of Fibonacci Series you want to print: 10
0, 1, 1, 2, 3, 5, 8, 13, 21, 34
```

Example # 4:

Derive logic to calculate total number of digits in a number.

Solution

```
#include<iostream>
using namespace std;
int main() {
    int n, count = 0;
    cout << "Enter number: ";
    cin >> n;

    for(int i = n; i > 0; i = i / 10) {
        count = count + 1;
    }

    cout << "Total Number of Digits: " << count;
    return 0;
}
```

The code produces the following output

```
Enter number: 4567
Total Number of Digits: 4
```

Challenge#1:

Print multiplication table of 24, 50 and 29 using loop.

TestCase:

```
C:\Users\ADMIN\Desktop>c++ fibo.cpp -o fibo.exe

C:\Users\ADMIN\Desktop>fibo.exe
Multiplication Table of 24:
24 x 1 = 24
24 x 2 = 48
24 x 3 = 72
24 x 4 = 96
24 x 5 = 120
24 x 6 = 144
24 x 7 = 168
24 x 8 = 192
24 x 9 = 216
24 x 10 = 240

Multiplication Table of 50:
50 x 1 = 50
50 x 2 = 100
50 x 3 = 150
50 x 4 = 200
50 x 5 = 250
50 x 6 = 300
50 x 7 = 350
50 x 8 = 400
50 x 9 = 450
50 x 10 = 500

Multiplication Table of 29:
29 x 1 = 29
29 x 2 = 58
29 x 3 = 87
29 x 4 = 116
29 x 5 = 145
29 x 6 = 174
29 x 7 = 203
29 x 8 = 232
29 x 9 = 261
29 x 10 = 290

C:\Users\ADMIN\Desktop>
```

Challenge#2:

Find the frequency of a digit in a number.

Test Cases:

frequencyChecker(566960, 6)	**3**
frequencyChecker(566960, 5)	**1**

```
C:\Users\ADMIN\Desktop>fibo.exe
Enter number: 467646477
Enter digit: 7
Frequency of 7 in 467646477 is: 3
C:\Users\ADMIN\Desktop>
```

```
C:\Users\ADMIN\Desktop>
C:\Users\ADMIN\Desktop>fibo.exe
Enter number: 57878485
Enter digit: 8
Frequency of 8 in 57878485 is: 3
C:\Users\ADMIN\Desktop>
```

Congratulations, you have performed all the tasks using the For Loop. Now let's move towards conditional Loops.

Conditional Loop:

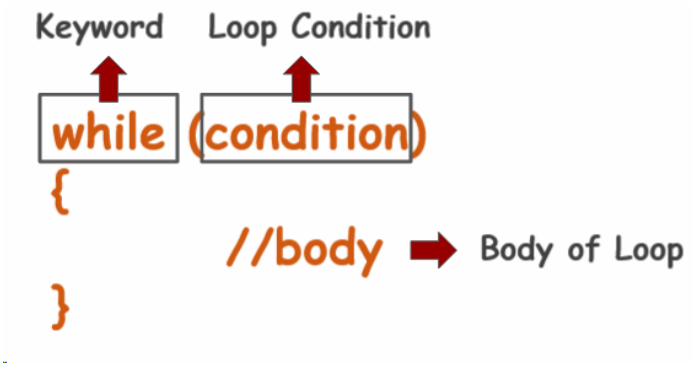
Conditional loops help to repeat a set of instructions until some condition is true. There are two common places for it use.

1. Reading an unknown amount of input from the user
2. Validating input.

C++ provides a while loop that is used as a conditional loop.

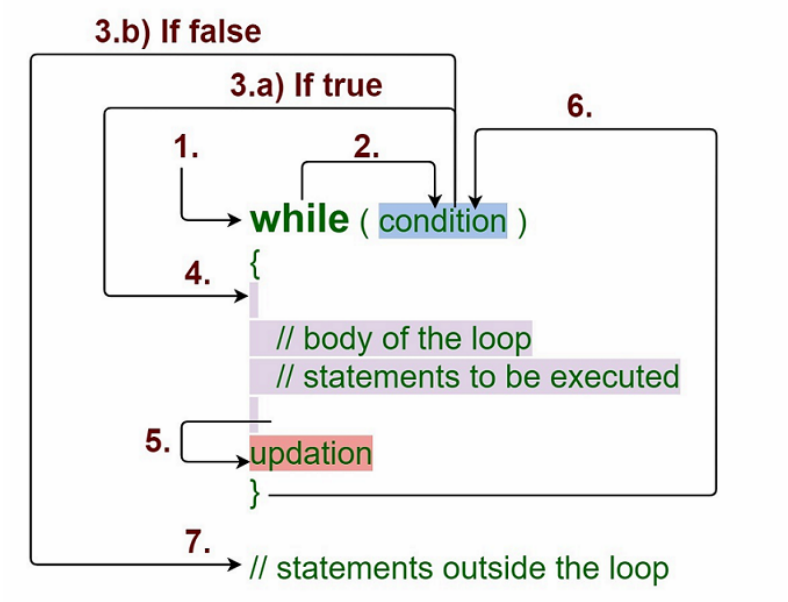
Syntax of While Loop

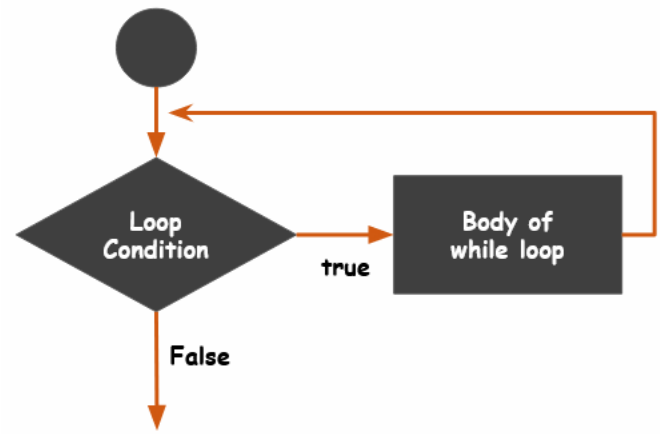
The syntax of a while loop in C++ is



The condition may be any expression, and true is any non-zero value. The loop iterates while the condition is true. When the condition becomes false, program control passes to the line immediately following the loop.

Flow of While Loop





Example #1: **Print a message "I am happy" until user presses "y" or any key to exit.**

Solution

```
#include <iostream>
using namespace std;
main()
{
    char ch = 'y';
    while(ch == 'y')
    {
        cout << "I am Happy" << endl;
        cout << "Press y to continue or any key to exit" << endl;
        cin >> ch;
    }
}
```

The code produces the following output

```
i am happy
Press y to continue or any key to exit
y
i am happy
Press y to continue or any key to exit
y
i am happy
```

Let's understand how we can validate input using a while loop with a working example.

Example #2:

Suppose the requirement is to enter a positive number but if the user enters negative number then an error message is shown and the user is again asked to enter a positive number.

Solution

```
#include <iostream>
using namespace std;
main()
{
    int value;
    cout << "Please enter a Positive Number: ";
    cin >> value;
    while (value <= 0)
    {
        cout << "Error: " << value << " is not a Positive Number." << endl;
        cout << "Please enter a Positive Number: ";
        cin >> value;
    }
    cout << "Program Ends" << endl;
}
```

The code produces the following output

```
C:\C++>c++ example.cpp -o example.exe

C:\C++>example.exe
Please enter a Positive Number: -3
Error: -3 is not a Positive Number.
Please enter a Positive Number: -70
Error: -70 is not a Positive Number.
Please enter a Positive Number: 9
Program Ends

C:\C++>
```

```
C:\Users\ADMIN\Desktop>c++ fibo.cpp -o fibo.exe

C:\Users\ADMIN\Desktop>fibo.exe
Please enter a Positive Number: -54678
Error: -54678 is not a Positive Number.
Please enter a Positive Number: -12345678
Error: -12345678 is not a Positive Number.
Please enter a Positive Number: -9
Error: -9 is not a Positive Number.
Please enter a Positive Number: 8
Program Ends
```

```
C:\Users\ADMIN\Desktop>fibo.exe
Please enter a Positive Number: 1
Program Ends

C:\Users\ADMIN\Desktop>
```

There are many problems that you can solve with both for loop or while loop. It's up to you which loop you want to use in which you are more comfortable.

Let's see the first example of for loop.

Example # 3:

Display counting from 1 to 10.

Solution with For Loop	Solution with While Loop

`#include <iostream>`
`using namespace std;`
`main()`
`{`
`  for(int i = 1; i<=10; i= i+1)`
`  {`
`  cout << i << endl;`
`  }`
 	
`}`
	`#include <iostream>`
	`using namespace std;`
	`main()`
	`{`
	`  int i = 1;`
	`  while (i <= 10)`
	`  {`
	`  cout << i << endl;`
	`  i = i + 1;`
	`  }`
	`}`

The code produces the following output	
1 2 3 4 5 6 7 8 9 10	

Example #4:

Calculate the sum of first 5 natural numbers.

Remember: Before writing the program, first step is to identify the pattern within the sub problem such that this pattern can be written as a generic expression or set of generic expressions.

Solution with For Loop	Solution with While Loop

`#include <iostream>`
`#include<iostream>`
`using namespace std;`
`main()`
`{`
`  int sum = 0;`
`  for(int i = 1; i<=5; i= i+1)`
`  {`
`  sum = sum + i;`
`  }`
`  cout << sum << endl;`
`}`
	`#include <iostream>`
	`#include <iostream>`
	`using namespace std;`
	`main()`
	`{`
	`  int i = 1, sum = 0;`
	`  while (i <= 5)`
	`  {`
	`  sum = sum + i;`
	`  i = i + 1;`
	`  }`
	`  cout << sum << endl;`
	`}`

The code produces the following output	
15	

Congratulations, you have practiced and learned the fundamental concepts of For and While Loop. Let's start the challenges now. You can use any loop.

Challenge 1:

Write two separate functions to find greatest common divisor (GCD) or highest common factor (HCF) of given two numbers.

Greatest Common Divisor (GCD) or Highest Common Factor (HCF) of two positive integers is the largest positive integer that divides both numbers without remainder.

Least Common Multiple (LCM) of two integers is the smallest integer that is a multiple of both numbers.

Hint:

LCM(a, b) = (a * b) / GCD(a, b)

TestCases:

```
C:\Users\ADMIN\Desktop>fibo.exe
Enter first number: 8
Enter second number: 12
GCD of 8 and 12 is: 4
LCM of 8 and 12 is: 24
```

```
C:\Users\ADMIN\Desktop>fibo.exe
Enter first number: 9
Enter second number: 13
GCD of 9 and 13 is: 1
LCM of 9 and 13 is: 117
```

Nested For Loop:

A **nested loop** has one loop inside of another. These are typically used for working with two dimensions such as printing stars in rows and columns as shown below. When a loop is nested inside another loop, the inner loop runs many times inside the outer loop. In each iteration of the outer loop, the inner loop will be re-started. The inner loop must finish all of its iterations before the outer loop can continue to its next iteration.

```
for ( initialization; condition; update statement) {

   for ( initialization; condition;update statement ) {

      // statement of inside loop
   }

   // statement of outer loop
}
```

Example #1 Write a program in C++ to display the pattern like right angle triangle using an asterisk.

```
*
* *
* * *
* * * *
* * * * *
```

Solution
With Nested For

```
#include <iotsream>
using namespace std;
main()
{
   int i,j,rows;
   cout<<"Input number of rows : "<<endl;
   cin>>rows;
   for(i=1;i<=rows;i++)
```

The code produces the following output
Number of rows : 10 *
The code produces the following output

Challenge#1: Write a program in C++ to display the pattern like diamond using an asterisk.

```
*
**
***
```

```
****
*****
*****
****
***
**
*
```

Challenge#2: Write a **C++ program** to create a simple **University Management System** using basic programming concepts.

Your program must include the following requirements:

1. The program should allow the user **3 login attempts** using a **for loop**.
2. The correct login credentials are:
   - **Username:** `admin`
   - **Password:** `1234`
3. If the username and password are correct using a **logical operator (&&)**, the program should display **"Login Successful"** and show the main menu.
4. After successful login, display the following menu:
   - **1 → Add Student**
   - **2 → View Student**
   - **3 → Add Course**
   - **4 → Exit**
5. When the user selects **Add Student**, the program should ask for **student name and age** and store them temporarily in variables.
6. When the user selects **View Student**, the program should display the stored student information.
7. When the user selects **Add Course**, the program should ask for a **course name** and store it temporarily.
8. If the user selects **Exit**, the program should terminate.
9. The program must use:
   - **if–else statements**
   - **logical operators (&&, ||, !)**
   - **for loop**

Code

`#include <iostream>`
`using namespace std;`
 	
`int main()`
`{`

```
    string username, password;

    string studentName = "";
    int studentAge = 0;
    string courseName = "";

    int choice;


    for(int i = 1; i <= 3; i++)
    {
        cout << "Enter Username: ";
        cin >> username;

        cout << "Enter Password: ";
        cin >> password;

        if(username == "admin" && password == "1234")
        {
            cout << "Login Successful"<<endl;
            break;
        }
        else
        {
            cout << "Wrong Login"<<endl;
        }

        if(i == 3 && !(username == "admin" && password == "1234"))
        {
            cout << "Too many attempts. Program End."<<endl;
            return 0;
        }
    }

    for(int i = 1; i <= 5; i++)
    {
        cout << endl<<"---- University Management System ----"<<endl;
        cout << "1. Add Student"<<endl;
        cout << "2. View Student"<<endl;
        cout << "3. Add Course"<<endl;
        cout << "4. Exit"<<endl;

        cout << "Enter Choice: ";
        cin >> choice;

        if(choice == 1)
        {
            cout << "Enter Student Name: ";
            cin >> studentName;

            cout << "Enter Student Age: ";
            cin >> studentAge;

            cout << "Student Added Successfully"<<endl;
        }
        else if(choice == 2)
        {
```

```
            if(studentName != "")
            {
                cout << "Student Name: " << studentName << endl;
                cout << "Student Age: " << studentAge << endl;
            }
            else
            {
                cout << "No Student Record Found"<<endl;
            }
        }
        else if(choice == 3)
        {
            cout << "Enter Course Name: ";
            cin >> courseName;

            cout << "Course Added: " << courseName << endl;
        }
        else if(choice == 4)
        {
            cout << "Program Exit"<<endl;
            break;
        }
        else
        {
            cout << "Invalid Choice"<<endl;
        }
    }


}
```

The code produces the following output

```
C:\Users\ADMIN\Desktop>c++ fibo.cpp -o fibo.exe

C:\Users\ADMIN\Desktop>fibo.exe
Enter Username: admin
Enter Password: 1234
Login Successful

---- University Management System ----
1. Add Student
2. View Student
3. Add Course
4. Exit
Enter Choice: 1
Enter Student Name: ali
Enter Student Age: 19
Student Added Successfully

---- University Management System ----
1. Add Student
2. View Student
3. Add Course
4. Exit
Enter Choice: 1
Enter Student Name: ahmad
Enter Student Age: 17
Student Added Successfully

---- University Management System ----
1. Add Student
2. View Student
3. Add Course
4. Exit
Enter Choice: 2
Student Name: ahmad
Student Age: 17

---- University Management System ----
1. Add Student
2. View Student
3. Add Course
4. Exit
Enter Choice: 4
Program Exit

C:\Users\ADMIN\Desktop>
```

Challenge#3: Write a **C++ program** to simulate a simple **ATM Management System** using basic programming concepts.

Requirements:

1. The program should allow the user **3 attempts to enter the ATM PIN** using a **for loop**.
2. The correct **PIN is 1234**.
3. If the correct PIN is entered using a **logical operator (==)**, the program should display **"Login Successful"** and show the ATM menu.
4. After successful login, display the following options:
   - **1 → Check Balance**
   - **2 → Deposit Money**
   - **3 → Withdraw Money**
   - **4 → Exit**

5. If the user selects **Check Balance**, the program should display the current account balance.
6. If the user selects **Deposit Money**, the program should ask for an amount and add it to the balance.
7. If the user selects **Withdraw Money**, the program should ask for an amount and deduct it from the balance **only if sufficient balance is available**.
8. If the balance is insufficient, display **"Insufficient Balance"**.
9. If the user selects **Exit**, the program should terminate.
10. The program must use:

- **if–else statements**
- **logical operators (&&, ||)**
- **for loop**

TestCase:

```
C:\Users\ADMIN\Desktop>fibo.exe
Enter ATM PIN: 6486
Wrong PIN
Enter ATM PIN: 4669
Wrong PIN
Enter ATM PIN: 84698
Wrong PIN
Too many attempts. Card Blocked.

C:\Users\ADMIN\Desktop>fibo.exe
Enter ATM PIN: 1234
Login Successful

------ ATM MENU ------
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Enter Choice: 1
Current Balance: 1000

------ ATM MENU ------
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Enter Choice: 2
Enter Amount to Deposit: 890
Money Deposited Successfully

------ ATM MENU ------
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Enter Choice: 3
Enter Amount to Withdraw: 280
Please collect your cash

------ ATM MENU ------
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Enter Choice: 1
Current Balance: 1610

------ ATM MENU ------
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Exit
Enter Choice: 4
Thank you for using ATM

C:\Users\ADMIN\Desktop>
```

Congratulations, you have practiced a lot and now have complete knowledge of loops.